



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Название: Обработка деревьев

Дисциплина: Типы и структуры данных

Вариант: 18

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Силантьева А.В.

Оглавление

Условие задачи	3
Описание ТЗ.....	3
1. Описание входных данных	3
2. Описание результатов работы программы.....	3
3. Способ обращения к программе	3
4. Задача программы	4
5. Описание возможных аварийных ситуаций и ошибок пользователя....	4
Описание внутренних структур данных	4
Описание алгоритма работы	5
Сравнение эффективности поиска узла двоичного дерева в зависимости от степени ветвления	5
Выводы	6
Ответы на контрольные вопросы	7

Условие задачи

В файловой системе каталог файлов организован в виде бинарного дерева. Каждый узел обозначает файл, содержащий имя и атрибуты файла, в том числе и дату последнего обращения к файлу. Написать программу, которая обходит дерево и удаляет из него все файлы, последнее обращение к которым происходило до определенной даты. Вывести исходное и измененное деревья в виде дерева.

Описание ТЗ

1. Описание входных данных

Ограничения:

1. Любой код, не лежащий в диапазоне от 0 до 7 включительно, считается ошибочным

2. Описание результатов работы программы

В результате работы программы создается двоичное дерево. После этого выводится меню программы, содержащее в себе следующие пункты:

- 0) Выход
- 1) Вывести дерево
- 2) Обход дерева
- 3) Включение узла в дерево
- 4) Удаление узла из дерева
- 5) Поиск узла в дереве
- 6) Обход дерева и удаление из него всех файлов, последнее обращение к которым происходило до определенной даты
- 7) Сравнить эффективность алгоритма поиска в зависимости от степени его ветвления

3. Способ обращения к программе

Обращение к программе происходит из командной строки путем запуска

исполняемого файла «*app.exe*».

4. Задача программы

Задача данной программы заключала в себе несколько составных частей:

- Обход дерева и удаления из него информации о файлах, время изменения которых меньше введенного пользователем
- Измерение времени, необходимого для поиска определенного узла двоичного дерева

5. Описание возможных аварийных ситуаций и ошибок пользователя

- Некорректный код меню:
 - Код возврата: 1
 - Сообщение: «Некорректный код. Попробуйте снова.»
- Ошибка выделения памяти:
 - Код возврата: 2
 - Сообщение: «Не удалось выделить память. Попробуйте снова.»
- Ошибка при пустой очереди:
 - Код возврата: 3
 - Сообщение: «Дерево пустое. Сначала добавьте элементы в него и попробуйте снова.»
- Ошибка ввода элемента:
 - Код возврата: 4
 - Сообщение: «Не удалось получить элемент. Попробуйте снова.»

Описание внутренних структур данных

Тип данных, описывающий структуру для работы с двоичным деревом, в узлах которого хранятся символы:

```
typedef struct tree_node
{
    int index;           // Индекс узла

    char value;          // Значение узла
}
```

```

    struct tree_node *left; // Указатель на левого потомка
    struct tree_node *right; // Указатель на правого потомка
} tree_node_t;

```

Тип данных, описывающий структуру для работы с двоичным деревом, в узлах которого хранится информация о файлах:

```

typedef struct file_tree_node
{
    char filename[MAX_FILENAME_LEN + 1]; // Имя файла
    struct
    {
        int num; // Число
        int month; // Месяц
        int year; // Год
    } date;

    struct file_tree_node *left; // Указатель на левого потомка
    struct file_tree_node *right; // Указатель на правого потомка
} file_tree_node_t;

```

Тип данных, описывающий структуру для хранения результатов замерного эксперимента:

```

typedef struct measure
{
    long long unsigned time; // Время
    int branching; // Процент ветвления
    int mem; // Память
} measurement_table;

```

Описание алгоритма работы

1. Пользователю выводится меню программы с возможностью выбрать желаемый
2. Пользователь вводит желаемый пункт меню
3. Пока пользователь не введет «0» (пункт выхода из программы), ему будет предложено вводить номера меню и выполнять желаемые действия

Сравнение эффективности поиска узла двоичного дерева в зависимости от степени ветвления

Полученная таблица замеров:

BRANCHING (%)	TIME / MEM (1000 elems, mcs)	TIME / MEM (10000 elems, mcs)	TIME / MEM (25000 elems, mcs)	TIME / MEM (50000 elems, mcs)	TIME / MEM (100000 elems, mcs)
0	11 / 24000	149 / 240000	315 / 600000	725 / 1200000	3372 / 2400000
25	9 / 24000	105 / 240000	243 / 600000	631 / 1200000	1837 / 2400000
50	8 / 24000	91 / 240000	227 / 600000	543 / 1200000	1385 / 2400000
75	7 / 24000	96 / 240000	204 / 600000	457 / 1200000	1345 / 2400000
100	5 / 24000	51 / 240000	132 / 600000	313 / 1200000	1210 / 2400000

Выводы

В ходе работы были изучены способы и алгоритмы работы с деревьями, а также проведены замерные эксперименты, по результатам которых можно сделать вывод, что реализованный мной алгоритм выполняет поиск элемента быстрее при большей степени ветвления дерева, так как происходит отбрасывание большого количества элементов в случае нахождения узла.

Таким образом, можно сделать вывод, что при данной реализации поиска, программисту выгодно использовать такой алгоритм при большом ветвлении и поиске элемента с левой части дерева.

Ответы на контрольные вопросы

- 1) Дерево – это нелинейная структура данных, используемая для представления иерархических связей, имеющих отношение «один ко многим».
- 2) Память под представление деревьев выделяется аналогично спискам – по мере необходимости, под каждый узел
- 3)
 - N-арное дерево
 - сбалансированное дерево
 - бинарное дерево
 - бинарное дерево поиска
 - И т.д.
- 4) Обход дерева, включение, исключение и поиск узлов
- 5) Это бинарное дерево, в котором левые потомки каждого узла имеют ключи (значения), меньшие ключа данного узла, а правые – большие